

Flink vs Spark for UrbanPulse

Course: DSE ZG556 — Stream Processing and Analytics

Assignment: UrbanPulse Situated Learning · Task C, Item 12

Which engine fits which UrbanPulse job?

Dimension	Bus Bunching Detection	Ward Energy Aggregation
Best Engine	✓ Apache Flink	✓ Apache Spark
Why	Sub-second keyed state per bus pair	Micro-batch 15-min windows, batch-friendly
State Size	Small — ~600 bus positions in memory	Medium — 800 meters × 60 wards per window
Latency Requirement	< 5 min (alert before buses diverge)	15-min tumbling window (batch acceptable)
Recovery Time Objective	< 5 min (safety-critical)	< 30 min (report delay tolerable)
Operational Complexity	Low — single keyed stream	Low — standard micro-batch Spark job

Dimension 1 — State Size

Bus Bunching (Flink)

Flink maintains **keyed state per route_id**: a small dictionary of recent bus positions (600 buses × 40 routes = ~15 bus positions per route). This is tiny — fits comfortably in Flink's in-memory state backend (RocksDB not needed). Flink's `KeyedProcessFunction` stores exactly the data needed per key and evicts expired state automatically.

Spark equivalent would be costly: Spark Structured Streaming would need to maintain a large state store across micro-batches for all bus pairs, using `flatMapGroupsWithState` — much heavier because Spark was designed for partition-level state, not per-key millisecond-latency state.

Ward Energy Aggregation (Spark)

Spark aggregates 800 meters across 60 wards in 15-minute windows. The state per window is a simple sum/avg/max per ward — at most 60 rows of aggregated data. Spark's `groupBy + window()` handles this natively in one SQL-like expression with no custom state management code. The 45-minute watermark is a single configuration line.

Dimension 2 — Latency Requirement

Bus Bunching (Flink)

The bunching rule fires when two buses stay within 200 metres for **more than 5 minutes**. Flink processes each GPS event as it arrives — latency is milliseconds. This means the alert fires as soon as the 5-minute threshold is crossed, giving the control centre time to dispatch a bus before the situation worsens.

Spark would miss the window: Spark micro-batches run every 1–5 minutes. In a 5-minute bunching scenario, Spark might detect it at minute 6 or 7 — too late for the dispatcher to act during peak hour.

Ward Energy Aggregation (Spark)

Ward energy totals are used for **councillor dashboards** refreshed every 15 minutes — not for emergency response. A 15-minute Spark micro-batch perfectly matches this requirement. Using Flink here would add unnecessary complexity (custom timers, manual watermark management) for a job that does not need sub-second response.

Dimension 3 — Recovery Time Objective (RTO)

Bus Bunching (Flink) — RTO < 5 min

Flink checkpoints keyed state to local disk every 5 minutes. On restart, it restores from the last checkpoint and replays unprocessed Kafka messages — ready in under 5 minutes. The GPS stream's 24-hour Kafka retention means no events are lost during the recovery window. **The signal control system cannot wait 30 minutes for a restart** — Flink's fast checkpoint-based recovery is essential here.

Ward Energy Aggregation (Spark) — RTO < 30 min acceptable

If the Spark job crashes mid-window, it restarts from its Kafka checkpoint offset and reprocesses the current 15-minute window from scratch — a cold start takes 2–5 minutes. Even a 30-minute restart is acceptable: councillors check dashboards hourly, not second-by-second. Spark's simpler recovery model (no per-key state restoration) makes it more operationally predictable for this batch-style job.

Dimension 4 — Operational Complexity

Bus Bunching (Flink)

Flink's `KeyedProcessFunction` gives full control over per-key state, timers, and side outputs — but requires understanding the Flink state model. For UrbanPulse, this complexity is justified because the alternative (approximating bus bunching in Spark micro-batches) would require complex `flatMapGroupsWithState` code that is harder to maintain and less accurate.

Ward Energy Aggregation (Spark)

Spark's `groupBy + window + agg` SQL API is the simplest possible code for this job — 10 lines of PySpark SQL. Any data engineer familiar with SQL can read and modify it. No custom state management, no timer registration, no side outputs. The 45-minute watermark is a single `.withWatermark()` call. **Operational simplicity wins for a job that runs unattended 24×7 on a city server with a small IT team.**

Summary

Use Flink when you need low-latency per-entity state updates (bus pairs, junction cycles) where the alert window is measured in minutes or seconds.

Use Spark when you need reliable, auditable batch-style aggregations (ward totals, rolling averages) where a 15-minute window is acceptable and SQL familiarity matters more than microsecond latency.

In UrbanPulse, both engines coexist on the same Kafka bus — **they are complementary, not competing.**